



VISION AND LEARNING LAB

GIẢI THUẬT HỒI QUY

TS. Hoàng Quốc Việt



HUNG YEN UNIVERSITY OF TECHNOLOGY AND EDUCATION



Giới thiệu



Linear Regression



Decision Tree



Random Forest

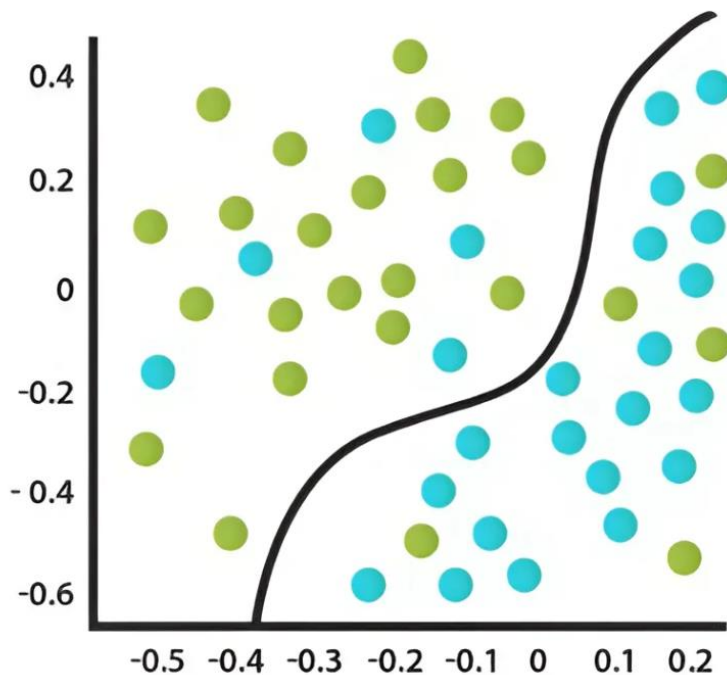


Gradient Boosting

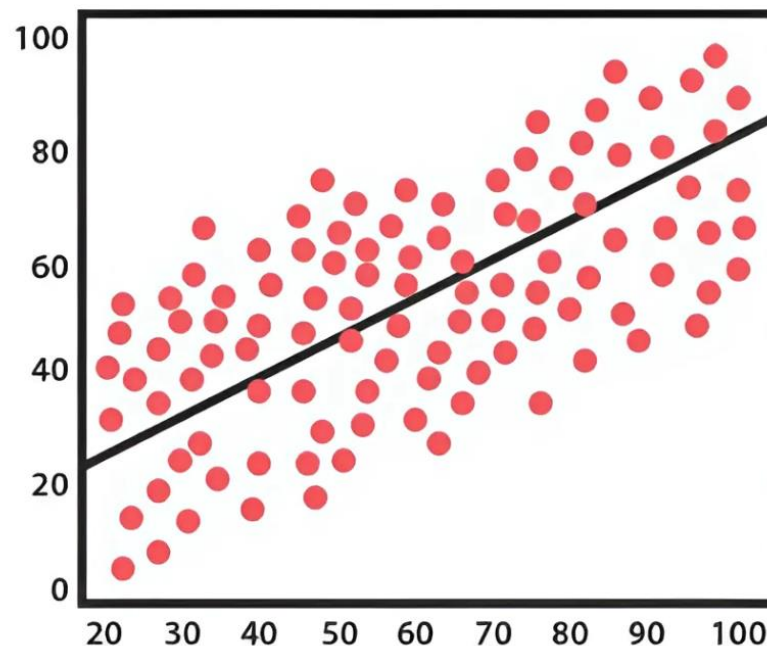


XGBoost

- Hai vấn đề cơ bản của học máy:



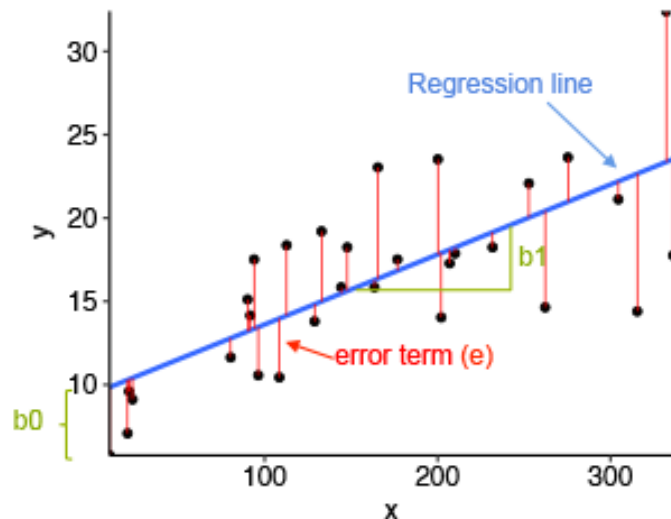
CLASSIFICATION



REGRESSION

- *Classification (phân loại)*: Một bài toán được gọi là classification nếu các label của input data được chia thành **một số hữu hạn nhóm**. Ví dụ: Dự đoán ảnh đầu vào là chó hay mèo.
- *Regression (hồi quy)*: Một bài toán được gọi là regression nếu các label không được chia thành các nhóm **mà một giá trị cụ thể**. Ví dụ: Một căn nhà có x phòng ngủ, có diện tích rộng y m² và cách trung tâm thành phố z km thì có giá bao nhiêu?

- Hàm loss/metric: Sử dụng hàm loss MSE , MAE , $RMSE$ (Root Mean Square Error), R^2 (R-Squared), $MAPE$ (Mean Absolute Percentage Error).



Mean Squared Error

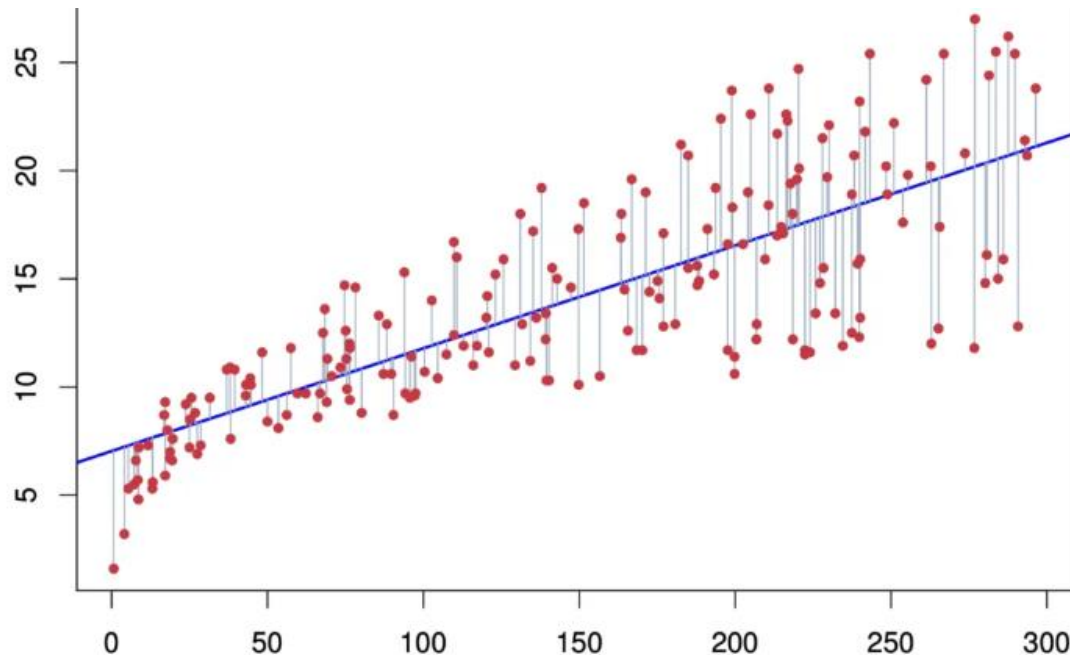
$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Mean Absolute Error

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

II. Linear Regression

- Linear Regression: Hồi quy tuyến tính.
- Linear Regression ước lượng một đường thẳng qua các điểm để dự báo tốt nhất xu hướng.



■ Ứng dụng:

- Dự đoán giá: dự đoán giá nhà, giá cổ phiếu dựa trên các yếu tố như vị trí, kích thước, chất lượng, lượng cung cầu, ...
- Dự đoán điểm số: dự đoán điểm số của học sinh dựa trên thời gian học, kỹ năng, trình độ giáo viên, ...
- Dự đoán sản phẩm: dự đoán đầu ra sản xuất dựa trên thời gian, công suất, nguyên liệu, lao động, ...
- Phân tích chuỗi thời gian: dự đoán xu hướng và chu kỳ của các chuỗi dữ liệu, như bất động sản, thời tiết, xu hướng sản xuất, ...
- ...

- Simple Linear Regression:
 - Mô hình này chỉ có một biến độc lập (input feature) mô tả mối quan hệ tuyến tính giữa biến phụ thuộc (output target) và biến độc lập.
 - Phương trình của Simple Linear Regression có dạng $y = a + bx + \epsilon$, trong đó a là điểm giao với trục tung (chỉ số độc lập), b là hệ số góc (độ dốc) của đường thẳng, và ϵ là sai số (y : Biến phụ thuộc, x : Biến độc lập).
- Ví dụ:
 - Dự đoán *chiều cao* của một người dựa vào *tuổi* của người đó (Biến độc lập X: Tuổi, biến phụ thuộc Y: Chiều cao).
 - Dự đoán năng suất cây trồng dựa vào lượng mưa.
 - ...

■ Multiple Linear Regression:

- Mô hình này có nhiều hơn một biến độc lập, biểu diễn mối quan hệ tuyến tính giữa các biến độc lập và biến phụ thuộc.
- Phương trình của Multiple Linear Regression có dạng $y = a + b_1x_1 + b_2x_2 + \dots + b_nx_n + \epsilon$, trong đó a là điểm giao với trục tung, $b_1, b_2, \dots, b_n$ là các hệ số góc, và ϵ là sai số (y : Biến phụ thuộc, các x_n : Biến độc lập).

■ Ví dụ:

- Dự đoán giá một ngôi nhà dựa vào thông tin ngôi nhà đó: Số phòng ngủ, diện tích ngôi nhà, khoảng cách đến thành phố...

- Simple Linear Regression:
 - Mô tả: Dự đoán lương của một nhân viên dựa trên năm kinh nghiệm của nhân viên đó.
 - Dataset:

	A	B
1	YearsExperience	Salary
2	1.1	39343
3	1.3	46205
4	1.5	37731
5	2	43525
6	2.2	39891
7	2.9	56642
8	3	60150
9	3.2	54445
10	3.2	64445

■ Code:

```
# 1. XỬ LÝ DỮ LIỆU
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# 1.1. Đọc file CSV
dataset = pd.read_csv('Dataset/Salary_Data.csv')
X = dataset.iloc[:, :-1].values
y = dataset.iloc[:, 1].values

# 1.2. Chia dữ liệu
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 1/3,
random_state = 0)

# Hiển thị số bản ghi tập train/test
print("Number of Training samples:", len(X_train))
print("Number of Testing samples:", len(X_test))
```

■ Code:

```
# 2. GIẢI THUẬT LINEAR REGRESSION
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, mean_absolute_error

# 1. Xử lý dataset: ở trên

# 2. Khởi tạo và huấn luyện mô hình Linear Regression
regressor = LinearRegression()
regressor.fit(X_train, y_train)

# 3. Dự đoán
y_pred = regressor.predict(X_test)

# 4. Đánh giá mô hình
mse = mean_squared_error(y_test, y_pred)
mae = mean_absolute_error(y_test, y_pred)
print("Mean Square Error (MSE): ", mse)
print("Mean Absolute Error (MAE): ", mae)
```

■ Code:

```
# 3. VISUALIZATION DATA
# 3.1. Visualising the Training set results
plt.scatter(X_train, y_train, color = 'red')
plt.plot(X_train, regressor.predict(X_train), color = 'blue')
```

```
# 3. VISUALIZATION DATA
# 3.2. Visualising the Test set results
plt.scatter(X_test, y_test, color = 'red')
plt.plot(X_train, regressor.predict(X_train), color = 'blue')
plt.title('Salary vs Experience (Test set)')
plt.xlabel('Years of Experience')
plt.ylabel('Salary')
plt.show()
```

■ Multiple Linear Regression

- Mô tả: Dự đoán giá nhà ở thành phố Boston.
- Dataset:
 - ❑ CRIM: per capita crime rate by town (Tỷ lệ tội phạm bình quân đầu người theo thị trấn)
 - ❑ ZN: proportion of residential land zoned for lots over 25,000 sq.ft.
 - ❑ INDUS: proportion of non-retail business acres per town.
 - ❑ CHAS: Charles River dummy variable (1 if tract bounds river; 0 otherwise)
 - ❑ NOX: nitric oxides concentration (parts per 10 million)
 - ❑ RM: average number of rooms per dwelling
 - ❑ AGE: proportion of owner-occupied units built prior to 1940
 - ❑ DIS: weighted distances to five Boston employment centres
 - ❑ RAD: index of accessibility to radial highways
 - ❑ TAX: full-value property-tax rate per \$10,000
 - ❑ PTRATIO: pupil-teacher ratio by town
 - ❑ $B - 1000(B_k - 0.63)^2$ where B_k is the proportion of blacks by town
 - ❑ LSTAT: % lower status of the population
 - ❑ MEDV: Median value of owner-occupied homes in \$1000's

■ Code:

```
# 1. XỬ LÝ DỮ LIỆU
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# 1.1. Đọc dữ liệu
data = pd.read_csv("Dataset/Boston.csv")
# dataset = pd.read_csv('Dataset/diabetes.csv')

# 1.2. Tách đặc trưng (X) và nhãn (y)
X = data.iloc[:, :-1]
y = data.iloc[:, -1]

# 1.3. Chia train/test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

# 1.4. Chuẩn hóa dữ liệu
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.fit_transform(X_test)

# Hiển thị số bản ghi tập train/test
print("Number of Training samples:", len(X_train))
print("Number of Testing samples:", len(X_test))
```

■ Code:

```
# 2. GIẢI THUẬT LINEAR REGRESSION
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, mean_absolute_error

# 1. Xử lý dataset: ở trên

# 2. Khởi tạo và huấn luyện mô hình Linear Regression
regressor = LinearRegression()
regressor.fit(X_train, y_train)

# 3. Dự đoán
y_pred = regressor.predict(X_test)

# 4. Đánh giá mô hình
mse = mean_squared_error(y_test, y_pred)
mae = mean_absolute_error(y_test, y_pred)
print("Mean Square Error : ", mse)
print("Mean Absolute Error : ", mae)
```

II. Linear Regression

■ Code:

```
# 5. Plotting Scatter graph to show the prediction
# results - 'ytrue' value vs 'y_pred' value
plt.scatter(y_test, y_pred, c = 'green')
plt.xlabel("Price: in $1000's")
plt.ylabel("Predicted value")
plt.title("True value vs predicted value : Linear Regression")
plt.show()
```

III. Decision Tree

- Decision Tree:

■ Code:

```
# 3. GIẢI THUẬT DECISION TREE
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor,
GradientBoostingRegressor
from sklearn.metrics import mean_squared_error, r2_score

# 1. Xử lý dataset: ở trên

# 2. Khởi tạo và huấn luyện mô hình Decision Tree
model_dt = DecisionTreeRegressor(max_depth=5, random_state=42)
model_dt.fit(X_train, y_train)

# 3. Dự đoán
y_pred = model_dt.predict(X_test)

# 4. Đánh giá
print("R2 Score:", r2_score(y_test, y_pred))
print("RMSE:", mean_squared_error(y_test, y_pred, squared=False))
```

IV. Random Forest

- Random Forest:

■ Code:

```
# 4. GIẢI THUẬT RANDOM FOREST
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor,
GradientBoostingRegressor
from sklearn.metrics import mean_squared_error, r2_score

# 1. Xử lý dataset: ở trên

# 2. Khởi tạo và huấn luyện mô hình Random Forest
model_rf = RandomForestRegressor(n_estimators=100, random_state=42)
model_rf.fit(X_train, y_train)

# 3. Dự đoán
y_pred = model_rf.predict(X_test)

# 4. Đánh giá
print("R2 Score:", r2_score(y_test, y_pred))
print("RMSE:", mean_squared_error(y_test, y_pred, squared=False))
```

- Gradient Boosting:

■ Code:

```
# 4. GIẢI THUẬT GRADIENT BOOSTING
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor,
GradientBoostingRegressor
from sklearn.metrics import mean_squared_error, r2_score

# 1. Xử lý dataset: ở trên

# 2. Khởi tạo và huấn luyện mô hình Gradient Boosting
model_gb = GradientBoostingRegressor(n_estimators=100, random_state=42)
model_gb.fit(X_train, y_train)

# 3. Dự đoán
y_pred = model_gb.predict(X_test)

# 4. Đánh giá
print("R2 Score:", r2_score(y_test, y_pred))
print("RMSE:", mean_squared_error(y_test, y_pred, squared=False))
```

- XGBoost:

■ Code:

```
# 5. GIẢI THUẬT XGBOOST
# Chú ý cài thư viện: pip install xgboost
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from xgboost import XGBRegressor
from sklearn.metrics import mean_squared_error, r2_score

# 1. Xử lý dataset: ở trên

# 2. Khởi tạo và huấn luyện XGBoost
model_xgb = XGBRegressor(
    n_estimators=200,      # số cây boosting
    learning_rate=0.1,     # tốc độ học
    max_depth=4,           # độ sâu mỗi cây
    subsample=0.8,         # lấy mẫu ngẫu nhiên
    colsample_bytree=0.8,  # lấy mẫu cột
    random_state=42
)
model_xgb.fit(X_train, y_train)

# 3. Dự đoán
y_pred = model_xgb.predict(X_test)

# 4. Đánh giá
print("R2 Score:", r2_score(y_test, y_pred))
print("RMSE:", mean_squared_error(y_test, y_pred, squared=False))
```

- [1]. <https://medium.com/>
- [2]. <https://towardsdatascience.com/>
- [3]. <https://machinelearningcoban.com/>
- [4]. Huang, Shih-Chia, and Trung-Hieu Le. *Principles and labs for deep learning*. Academic Press, 2021.

Thank You !

Question?